Night Before (Tuesday Night)

0 Prep Checklist — 30 min

- [ ] Confirm client has ready: Wi-Fi password, Anthropic API key (`console.anthropic.com`), Telegram installed, Google account login
- [ ] Pack: HDMI dummy plug, USB-C hub, keyboard + mouse, laptop, phone charger
- [ ] Review gog (Google Workspace) setup: `youtube.com/watch?v=Z0tfD0wJt5M`
- [ ] Read through this document end to end

On-Site Installation

1 Plug In + Connect — 10 min

- [ ] Place Mac Mini in preferred location (near power + internet)
- [ ] Plug in power, connect temporary monitor + keyboard
- [ ] Connect to client's Wi-Fi
- [ ] Open Terminal (Cmd+Space, type "Terminal")
- [ ] Test internet: `ping google.com` (Ctrl+C to stop)

2 Install Everything

15 min

- [ ] **Install Homebrew** (the Mac package manager)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- [ ] **Add Homebrew to PATH** (copy the "Next steps" it prints, or run this):

```
echo >> ~/.zprofile
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

- [ ] **Install Node 22**

```
brew install node@22
```

- [ ] **Fix PATH** (so openclaw, claude, and other global commands are found):

```
echo 'export PATH="/usr/local/bin:/opt/homebrew/bin:$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

- [ ] **Install OpenClaw**

```
sudo npm install -g openclaw
```

- [ ] **Install Google tools + remote access + Telegram**

```
brew install steipete/tap/gogcli
brew install --cask tailscale
brew install --cask telegram
```

- [ ] **Install Claude Code**

```
sudo npm install -g @anthropic-ai/claude-code
claude install
```

> Running `claude` the first time adds it to the command line and starts the login flow.

- [ ] **Claude login + setup token**

```
claude /login
```

> Follow the browser OAuth flow to authenticate Claude with the client's Anthropic account.

```
claude setup-token
```

> Generates a setup token for OpenClaw to use Claude as the backend model. Copy and save this token — you'll need it during `openclaw onboard`.

- [ ] **Keep Mac awake permanently** (no App Store needed):

```
caffeinate -s &
```

> To make this persist after reboots, add `caffeinate -s &` to a Login Item or launchd plist. For now, running it once is fine.

- [ ] **Configure energy settings**:
  - [ ] System Settings > Energy > Turn display off: **Never**
  - [ ] System Settings > General > Startup & Login > "Start up automatically after power failure": **ON**
- [ ] **Enable Remote Login (SSH)**: System Settings > General > Sharing > Remote Login > **ON**
- [ ] **Open Tailscale**, have client sign in or create a free account. Note the Tailscale IP (`100.x.x.x`)

> **Checkpoint:** Run `openclaw --version`, `gog --version`, `claude --version`. All should print version numbers. Tailscale connected, energy settings configured, SSH enabled.

3 Configure OpenClaw

10 min

- [ ] Run the setup wizard:

```
openclaw onboard --install-daemon
```

- [ ] When it asks for provider: choose **Anthropic**
- [ ] Paste **client's API key** (from console.anthropic.com)
- [ ] Choose model: `claude-opus-4`
- [ ] Finish the wizard (accept defaults for anything you're unsure about)
- [ ] Check it's running:

```
openclaw status
```

> **Checkpoint:** Status shows "running". If not, run `openclaw gateway start`

3b Optional: Second OpenClaw Profile (Same Mac)

8 min

- [ ] If the client wants a separate OpenClaw setup for a spouse or second user, create a **second profile** instead of mixing both people into one install.
- [ ] Run a second onboard flow using a profile name like `wife`:

```
openclaw --profile wife onboard
```

- [ ] Choose a **different base port** from the main install. Leave at least **20 ports** between the two setups so there are no collisions.

> **Example:** Main profile on `18789`, second profile on `18820` or `18900`.

- [ ] Install and start the second profile as its own service:

```
openclaw --profile wife gateway install
openclaw --profile wife gateway start
```

- [ ] Check status:

```
openclaw --profile wife status
```

> **Why this matters:** Each profile gets separate config, state, sessions, and service management. That keeps each person's OpenClaw isolated on the same machine.

4 Telegram Bot

10 min

- [ ] On client's phone, open Telegram, search `@BotFather`
- [ ] Send `/newbot`
- [ ] Pick a name (e.g. "Greg's Assistant")
- [ ] Pick a username ending in `bot` (e.g. `gregtracy_bot`)
- [ ] BotFather gives you a token. Copy it.
- [ ] Add token to OpenClaw:

```
openclaw config
```

> The config wizard will walk you through adding the Telegram token. If you prefer manual: `nano ~/.openclaw/openclaw.json` and add `channels.telegram.enabled: true`, `channels.telegram.botToken: "TOKEN"`, `channels.telegram.dmPolicy: "pairing"`

- [ ] Restart:

```
openclaw gateway restart
```

- [ ] Client searches for the bot in Telegram and sends any message
- [ ] A pairing code appears. Approve it:

```
openclaw pairing list telegram
openclaw pairing approve telegram CODE_HERE
```

- [ ] Client sends "Hello". AI responds.

> **Checkpoint:** Client is chatting with their AI on Telegram.

4b iMessage Support

10 min

- [ ] Make sure **Messages app** is signed in with client's Apple ID on the Mac
- [ ] **Install imsg**:

```
brew install steipete/tap/imsg
```

- [ ] **Trigger macOS permission prompts** (run interactively):

```
imsg chats --limit 1
```

> macOS will ask for **Full Disk Access** and **Automation** permissions. Approve both. If prompts don't appear, go to System Settings > Privacy & Security and add Terminal (or whatever app is running this) manually.

- [ ] **Add iMessage to OpenClaw config**:

```
openclaw config
```

> If the wizard doesn't cover iMessage, edit manually: `nano ~/.openclaw/openclaw.json`

```
{
  "channels": {
    "imessage": {
      "enabled": true,
      "cliPath": "/opt/homebrew/bin/imsg",
      "dbPath": "/Users/USERNAME/Library/Messages/chat.db"
    }
  }
}
```

> Replace `USERNAME` with the actual Mac username. Find it with `whoami`.

- [ ] **Restart and pair**:

```
openclaw gateway restart
```

- [ ] Client sends an iMessage to the Mac's Apple ID (phone number or email)
- [ ] Approve the pairing:

```
openclaw pairing list imessage
openclaw pairing approve imessage CODE_HERE
```

- [ ] Client sends "Hello" via iMessage. AI responds.

> **Checkpoint:** Client is chatting with their AI via iMessage.

5 Google Workspace (Gmail, Calendar, Drive, Docs) 15 min

- [ ] **Create Google Cloud project** (open browser on the Mac Mini):
  - [ ] Go to `console.cloud.google.com`
  - [ ] Sign in with client's Google account
  - [ ] Click "Select a project" at the top, then "New Project"
  - [ ] Name it "AI Assistant" or whatever, click Create
- [ ] **Enable the APIs**:
  - [ ] Go to APIs & Services > Library
  - [ ] Search and enable: **Gmail API**, **Google Calendar API**, **Google Drive API**, **Google Docs API**
- [ ] **Create OAuth credentials**:
  - [ ] Go to APIs & Services > Credentials
  - [ ] Click "Configure Consent Screen" > External > fill in app name + your email > Save
  - [ ] Go to **OAuth consent screen > Audience** and switch publishing status to **"In production"** (removes test-user restriction)
  - [ ] Go back to Credentials > Create Credentials > OAuth client ID
  - [ ] Type: **Desktop app**, name it anything
  - [ ] Click **Download JSON**
- [ ] **Connect gog**:

```
gog auth credentials ~/Downloads/client_secret_*.json
gog auth add client@email.com --services gmail,calendar,drive,docs
```

> This opens a browser. Client signs in and approves. It will say "This app isn't verified" because it's a test app. Click "Advanced" > "Go to [app name]" > Allow all permissions.

- [ ] Test it:

```
gog calendar events primary --from 2026-02-18T00:00:00Z --to 2026-02-19T00:00:00Z
gog gmail search "newer_than:1d" --max 5
```

- [ ] Test on Telegram: Client asks "What's on my calendar today?"

> **Checkpoint:** AI can read Greg's calendar and email.

> **If this is taking too long on-site:** Skip the Google Cloud setup. Tell client: "I'll finish the Google connection tonight remotely. Everything else is live." Use Tailscale to SSH in and complete it later. Client wakes up to a fully working system.

6 Humanizer (SOUL.md) 10 min

- [ ] Create the personality file:

```
nano ~/.openclaw/workspace/SOUL.md
```

- [ ] Paste the following SOUL.md content:

```
# SOUL.md

## Identity
I'm Greg's personal AI assistant. I help with mortgage recruiting,
scheduling, content creation, and keeping his day organized.

## How I Talk
- Casual and direct. Like texting a smart colleague.
- Match Greg's energy. If he's quick and to the point, I am too.
- No corporate speak. No "I'd be delighted to assist." Just be normal.

## The Humanizer (CRITICAL)
When writing ANY content, messages, emails, posts, or outreach:
- Write like a real person, not an AI
- Use contractions naturally (I'm, don't, we're, can't)
- Mix up sentence length. Short ones. Then maybe a longer one
  that takes its time getting where it's going.
- No filler phrases like "In today's fast-paced world" or
  "Let's dive in" or "Here's the thing"
- No emoji walls. One emoji max, only if it fits.
- No hashtag spam. 2-3 max on social posts.
- If you read it out loud and it sounds like a robot, rewrite it.
- Sound like Greg would sound if he had 30 minutes to write it.

## Recruiting
- Greg is in mortgage recruiting. This is his main business.
- Draft outreach that sounds personal and genuine, not templated
- Research recruits when asked
- Always sound like Greg wrote it himself

## Morning Briefing
Every morning, send:
1. Today's schedule
2. Any messages or follow-ups that need attention
3. Quick weather check
4. Recruiting tasks or reminders

## Rules
- Don't make decisions for Greg, give him options
- Don't send messages as Greg without his explicit OK
- Be concise. Don't over-explain.
```

- [ ] Ask client: "Does this sound like your style? Anything you'd adjust?"
- [ ] Restart the gateway so it loads the new file:

```
openclaw gateway restart
```

- [ ] Test on Telegram: "Write me a LinkedIn post about mortgage recruiting"

> **Checkpoint:** Content sounds human, not robotic. This is what clients care about most.

6b Install Humanizer Skill (from ClawHub) 5 min

- [ ] Install the ClawHub CLI (skill package manager):

```
sudo npm install -g clawhub
```

- [ ] Install the humanizer skill:

```
npx clawhub install humanizer
```

> This downloads the skill to `~/.openclaw/skills/humanizer/`. It teaches the AI to detect and remove AI writing patterns: em dashes, filler phrases, promotional language, rule of three, etc. Based on Wikipedia's "Signs of AI writing" guide.

- [ ] Restart gateway so it loads the new skill:

```
openclaw gateway restart
```

- [ ] Verify it loaded:

```
openclaw skills
```

- [ ] Look for "humanizer" in the list with status "ready"
- [ ] **Test:** On Telegram, ask: "Write a LinkedIn post about mortgage recruiting, then humanize it"

> **Checkpoint:** Humanizer skill is installed. Combined with the SOUL.md rules, content will sound genuinely human.

6c Model Match Integration

15 min

- [ ] **What Model Match is:** Mortgage recruiting intelligence platform (modelmatch.com). Greg uses it to find loan officers and real estate agents to recruit.

> **What the research found:** Model Match does NOT have a traditional REST API for querying data. They support **webhook push** (Premium/Enterprise plans) and **Zapier**. This means Model Match can push contact data OUT to a URL when Greg selects contacts in the app. We set up OpenClaw to receive that data.

- [ ] **Option A: Webhook Integration (best option)**
- [ ] Check if client has Premium or Enterprise plan (webhooks require it)
- [ ] In Model Match, go to Settings > Integrations > "Push to Webhooks"
- [ ] We need a webhook URL for OpenClaw to receive the data. Set one up:

```
# OpenClaw can receive webhooks via its hooks system
# Check docs: /opt/homebrew/lib/node_modules/openclaw/docs/hooks/
# Or use a simple webhook relay like webhook.site for testing first
```

- [ ] Paste the webhook URL into Model Match's integration settings
- [ ] **How it works after setup:** Client finds prospects in Model Match, clicks "Push to Webhook", and the contact data (name, email, phone, company, loan volume, NMLS ID, etc.) gets sent directly to OpenClaw. The AI can then draft personalized outreach based on that data.

- [ ] **Option B: Browser Access (day-one fallback)**
- [ ] **Install Google Chrome** (if not already installed):

```
brew install --cask google-chrome
```

- [ ] **Install the OpenClaw Chrome extension**:

```
openclaw browser extension install
```

> Open Chrome > `chrome://extensions` > enable **Developer mode** (top right) > **Load unpacked** > select path from `openclaw browser extension path`

- [ ] **Start the OpenClaw browser**:

```
openclaw browser start
```

- [ ] **Log Greg into Model Match in the OpenClaw browser**:
  - [ ] Open Model Match in the automation browser:

```
openclaw browser open https://app.model-match.com
```

  - [ ] Take a snapshot to see the login page:

```
openclaw browser snapshot
```

  - [ ] Have Greg type his credentials into the browser (or use CLI):

```
# Find the email/username field ref from the snapshot, then:
openclaw browser type REF "greg@blueroof.com"
# Tab to password field or find its ref:
openclaw browser type REF "password_here"
# Click the sign-in button:
openclaw browser click REF
```

> Replace REF with the actual element refs from the snapshot. The browser saves cookies/session automatically — Greg won't need to log in again.

  - [ ] Verify the AI can see the logged-in dashboard:

```
openclaw browser snapshot
```

- [ ] **Add Model Match navigation guide to TOOLS.md**:

```
nano ~/.openclaw/workspace/TOOLS.md
```

Add a section so the AI knows how to use the site on its own:

```
### Model Match (app.model-match.com)
- Already logged in via OpenClaw browser (cookies persist)
- To open: use browser tool > navigate to app.model-match.com
- Dashboard: pipeline overview, recent activity
- Prospects tab: search/filter loan officers by state, volume, product type, NMLS ID
- Contact details: click any prospect for full profile (name, email, phone, company, loan data)
- Key actions: search prospects, view profiles, export contacts, push to webhook
```

- [ ] **Live test** — ask the AI on Telegram: "Look up [a prospect name] on Model Match"

The AI should open the browser, navigate to Model Match (already logged in), search for the prospect, and return the results. If it works, Model Match is fully set up.

- [ ] **Data Model Match sends via webhook:**

  FirstName, LastName, Email, Phone, Company, Title, NMLS ID, Address, Trailing 12-month loan count/volume, product mix (conventional/FHA/VA), buyer/seller counts, custom fields, tags, and Model Match profile URL.

Realistic plan for day one: Set up the browser login so the AI can access Model Match immediately. Then configure the webhook integration remotely during the support period (it needs a stable URL, which Tailscale + a small webhook server provides). Tell client: "Your AI can access Model Match right now through the browser. This week I'll set up the direct data pipeline so contacts push straight to your AI automatically."

Checkpoint: AI can access Model Match (browser), with webhook pipeline coming in week one.

7 Travel/Maps + Recruiting + Briefing — 5 min

- [ ] **Travel/Maps:** Already works. OpenClaw has web search built in. No setup.
- [ ] **Recruiting:** Already works via SOUL.md + Claude. No setup.
- [ ] **Morning briefing:** Ask Greg what time he wants it
- [ ] Quick tests on Telegram:
  - [ ] "How far from Draper to SLC airport?" (maps)
  - [ ] "Draft a message to recruit Sarah, loan officer at Wells Fargo for 5 years" (recruiting)

Checkpoint: Both work out of the box. No config needed.

8 Go Headless — 5 min

- [ ] Disconnect monitor, plug in HDMI dummy plug
- [ ] Test: AI still responds on Telegram without monitor
- [ ] Test from your laptop: `ssh username@100.x.x.x` (Tailscale IP from Phase 2)

Checkpoint: Headless, always-on, remotely accessible.

9 Demo + Handoff — 10 min

- [ ] Walk client through on his phone (let the client type these):
  - [ ] "What's on my calendar today?"
  - [ ] "Summarize my recent emails"
  - [ ] "Write a LinkedIn post about mortgage recruiting"
  - [ ] "Draft a message to recruit James, loan officer at Chase"
  - [ ] "How far from Draper to Park City?"
- [ ] Confirm his morning briefing time
- [ ] Tell him: "If anything stops working, text me. I can remote in and fix it."
- [ ] "I'll check in tomorrow to make sure everything is running great."

If Something Goes Wrong

Problem	What to Do
Bot not responding	`openclaw gateway restart`
Model error / API key issue	Double-check key at console.anthropic.com, re-paste if needed
Gog "unverified app" warning	Click "Advanced" > "Go to [app name]". This is normal for test apps.
Calendar shows no events	Make sure you used the right email in `gog auth add`
AI sounds robotic	Edit SOUL.md, make humanizer rules more specific
Something you can't figure out	Say "Let me finish this remotely tonight." Use Tailscale later.
Anything at all	Check logs: `openclaw logs --follow` (90% of answers are here)

> **Fallback:** If any step takes longer than expected, say: "I want to make sure this is done right, so I'll finish this part remotely tonight." Complete it via Tailscale.